

Unit-2 Word level processing

Topics:

➤Types of Morphology:

- English and Indian Languages,**
- Finite-State Morphological Parsing,**
- building a Finite-State Lexicon,**
- Finite-State Transducers, FSTs for Morphological Parsing,**
- Transducers and Orthographic rules,**
- The Porter Stemmer,**
- Word and Sentence Tokenization,**
- Detecting and Correcting Spelling Errors,**
- Minimum Edit Distance,**

➤Human Morphological Processing N-Grams:

- Building N-gram for spelling corrections,**
- N-gram for language model**

➤ **Types of Morphology:**

1. Inflectional Morphology:

Definition: Inflectional morphology involves changes to a word's form to indicate grammatical information, such as tense, number, gender, or case.

Example: In English, the verb "run" inflects to "ran" (past tense) or "running" (present participle).

2. Derivational Morphology:

Definition: Derivational morphology is concerned with creating new words by adding affixes (prefixes, suffixes) to a base word, often resulting in a change in meaning or lexical category.

Example: From the base word "happy," the addition of the suffix "-ness" forms the derived word "happiness."

3. Concatenative Morphology:

Definition: Concatenative morphology involves combining morphemes through concatenation, where affixes are added to a base morpheme to create a new word.

Example: In English, the word "happily" is formed by concatenating the base word "happy" with the suffix "-ly."

4. Non-concatenative Morphology:

Definition: Non-concatenative morphology involves changes within a morpheme without straightforward concatenation. This can include internal modifications, vowel changes, or consonant shifts.

Example: In Arabic, the root morpheme "k-t-b" can generate words like "kitab" (book), "kutub" (books), or "maktab" (office) through non-concatenative changes.

5. Agglutinative Morphology:

Definition: Agglutinative morphology is characterized by the addition of multiple affixes to a base morpheme in a clear and modular fashion, with each affix representing a specific grammatical function.

Example: In Turkish, the word "ev" (house) can be modified to "evlerinizden" (from your houses) by agglutinating suffixes for possession, plural, and location.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

6. Polysynthetic Morphology:

Definition: Polysynthetic morphology involves combining multiple morphemes into a single word, creating complex words that may represent entire sentences or clauses. Example: In Inuktitut, a polysynthetic language, the word "ᑎᑭᑭᑦᑕᑦᑭᑦᑕᑦᑭᑦ" ("tigpiqtaujukkut") means "Do you have any tobacco for sale?" and includes various morphemes within a single word.

7. Isolating Morphology:

Definition: Isolating morphology is characterized by a one-to-one correspondence between morphemes and words. Each morpheme typically carries a distinct meaning, and words are composed of unaltered morphemes.

Example: Mandarin Chinese is often considered an isolating language, where words are composed of individual morphemes, and changes in meaning are often conveyed through word order or context.

Understanding these types of morphology is crucial for NLP tasks, such as tokenization, stemming, and lemmatization, as they involve analyzing and processing words based on their morphological structures.

English and Indian languages present interesting challenges and opportunities in the field of Natural Language Processing (NLP) due to their linguistic diversity, morphological richness, and varied script systems. Let's explore key aspects related to English and Indian languages in NLP:

• **English and Indian Languages,**

English in NLP:

Tokenization and Part-of-Speech Tagging:

English has a relatively straightforward structure, making tokenization (breaking text into words) and part-of-speech tagging more straightforward compared to languages with complex morphologies.

Named Entity Recognition (NER):

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

English NER systems benefit from consistent capitalization rules, making it easier to identify entities like names, locations, and organizations.

Machine Translation:

English is often used as a source or target language in machine translation tasks due to the availability of large bilingual corpora.

Sentiment Analysis:

English sentiment analysis models are well-developed, given the abundance of labeled datasets and resources for training and evaluation.

Indian Languages in NLP:

Morphological Complexity:

Many Indian languages exhibit complex morphologies with intricate systems of prefixes, suffixes, and infixes. Analyzing and processing these morphologies pose challenges for tasks like stemming and lemmatization.

Script Diversity:

Indian languages employ various scripts, such as Devanagari, Tamil, Bengali, and more. NLP systems need to be script-aware for accurate processing.

Named Entity Recognition:

Named Entity Recognition in Indian languages faces challenges due to the variety of names, titles, and honorifics used, which may not follow a standardized format. Machine Translation Challenges:

Machine translation involving Indian languages can be challenging due to variations in sentence structures, the absence of word boundaries in some scripts, and differences in syntactic and semantic structures.

Code-Mixing and Code-Switching:

Many Indian users engage in code-mixing (using multiple languages within a sentence) and code-switching (switching between languages). NLP models need to account for such linguistic phenomena.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material
Resource Constraints:

Compared to English, Indian languages may have fewer labeled datasets, linguistic resources, and tools for training and evaluating NLP models, which can affect model performance.

Localization and Regional Dialects:

Indian languages often have regional dialects and variations. Localizing NLP models for specific regions requires considering these linguistic nuances.

Speech Recognition:

Speech recognition systems for Indian languages need to account for diverse accents, pronunciation variations, and the coexistence of multiple languages in a conversation.

Multilingual NLP:

Multilingual Models:

Recent advancements include the development of multilingual models that can handle several languages, including English and various Indian languages, within a single architecture.

Cross-lingual Transfer Learning:

Techniques like cross-lingual transfer learning aim to leverage knowledge gained from one language (often English) to improve NLP performance in languages with fewer resources. In summary, while English benefits from a wealth of NLP resources, Indian languages present unique challenges related to script diversity, morphological complexity, and resource constraints. Ongoing research aims to address these challenges and enhance the inclusivity of

NLP applications across a wide linguistic landscape.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- **building a Finite-State Lexicon,**

Natural language processing (NLP) comprises a number of tasks that deal with the processing of natural language through computers. The theory of automata plays a significant role in providing solutions to many problems in natural language processing. For example, speech recognition, spelling correction, information retrieval, etc. Finite state methods are pretty useful in processing natural language as the modeling of information using rules has many advantages for language modeling. Finite state automaton has a mathematical model which is quite understandable; data can be represented in a compacted form using finite state automaton and it allows automatic compilation of system components.

Finite state automata (deterministic and non-deterministic finite automata) provide decisions regarding the acceptance and rejection of a string while transducers provide some output for a given input. Thus the two machines are quite useful in language processing tasks. Finite state automata are useful in deciding whether a given word

belongs to a particular language or not.

Finite State Automata

The term automata, derived from the Greek word “αὐτόματα” meaning “self-acting”, is the plural of automaton which may be defined as an abstract self-propelled computing device that follows a predetermined sequence of operations automatically.

An automaton having a finite number of states is called a Finite Automaton (FA) or Finite State automata (FSA).

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Mathematically, an automaton can be represented by 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where –

- ▶ Q is a finite set of states.
- ▶ Σ is a finite set of symbols, called the alphabet of the automaton
- ▶ δ is the transition function
- ▶ q_0 is the initial state from where any input is processed ($q_0 \in Q$)
- ▶ F is a set of final state/states of Q ($F \subseteq Q$)

Types of Finite State Automation (FSA)

Finite state automation is of two types.

Deterministic Finite Automation (DFA)

It may be defined as the type of finite automation wherein, for every input symbol we can determine the state to which the machine will move. It has a finite number of states which is why the machine is called Deterministic Finite Automaton (DFA).

Mathematically, a DFA can be represented by 5-tuple $(Q, \Sigma, \delta, q_0,$

F), where –

- ▶ Q is a finite set of states.
- ▶ Σ is a finite set of symbols, called the alphabet of the automaton.
- ▶ δ is the transition function where $\delta: Q \times \Sigma \rightarrow Q$.
- ▶ q_0 is the initial state from where any input is processed ($q_0 \in Q$).
- ▶ F is a set of final state/states of Q ($F \subseteq Q$).

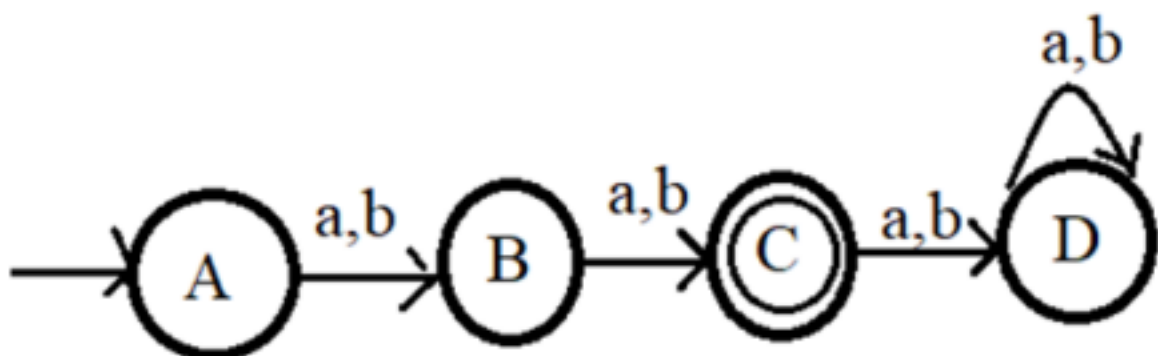
Whereas graphically, a DFA can be represented by diagrams called state diagrams where –

- ▶ The states are represented by vertices.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- ▶ The transitions are shown by labeled arcs.
- ▶ The initial state is represented by an empty incoming arc.
- ▶ The final state is represented by double circle.

Example of DFA –



Non-deterministic Finite Automation (NFA)

It may be defined as the type of finite automation where for every input symbol we cannot determine the state to which the machine will move i.e. the machine can move to any combination of the states. It has a finite number of states which is why the machine is called Non deterministic Finite Automation (NFA).

Mathematically, N DFA can be represented by 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where –

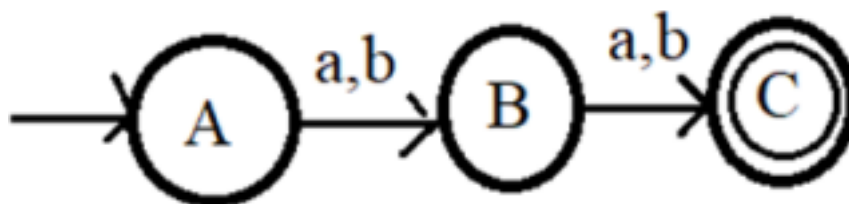
- ▶ Q is a finite set of states.
- ▶ Σ is a finite set of symbols, called the alphabet of the automaton.
- ▶ δ is the transition function where $\delta: Q \times \Sigma \rightarrow 2^Q$. ▶ q_0 is the initial state from where any input is processed ($q_0 \in Q$). ▶ F is a set of final state/states of Q ($F \subseteq Q$).

Whereas graphically (same as DFA), a N DFA can be represented by diagrams called state diagrams where –

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- ▶ The states are represented by vertices.
- ▶ The transitions are shown by labeled arcs.
- ▶ The initial state is represented by an empty incoming arc.
- ▶ The final state is represented by double circle.

Example of N DFA –



Finite Automata in NLP

Finite State Transducer

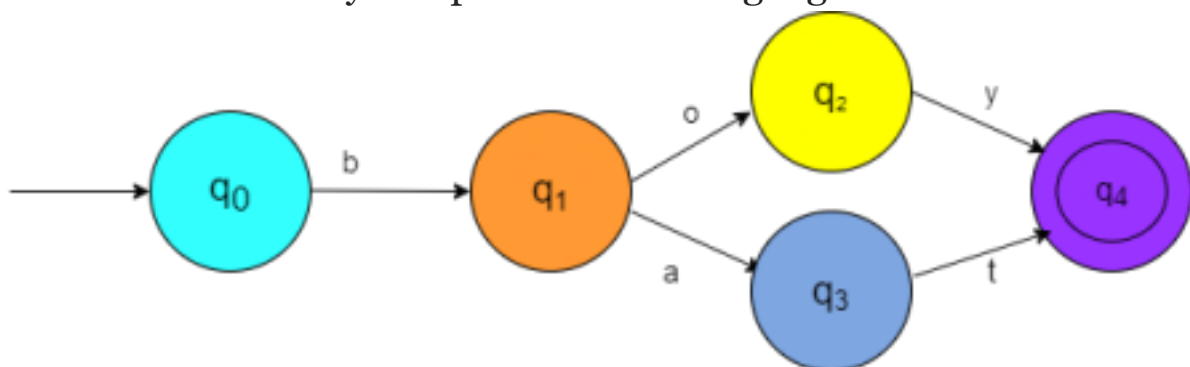
A finite-state transducer (FST) is a finite-state machine with two memory tapes, following the terminology for Turing machines: an input tape and an output tape. This contrasts with an ordinary finite state automaton, which has a single tape. An FST is a type of finite

state automaton (FSA) that maps between two sets of symbols. An FST is more general than an FSA. An FSA defines a formal language by defining a set of accepted strings, while an FST defines relations between sets of strings.

Language Recognizer

There are many tasks that need language recognizing mechanism. For example, spelling checker, morphological analysis, language identification etc.. Finite state machine are quite useful as a language recognizer. For a given word, a NFA can be designed easily that recognize the word. For example, NFA for the words 'boy' and 'bat' is

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material shown in the Figure below. Similarly for every word a NFA can be designed and the different NFA's can be combined to form spelling checker or dictionary compilation for a language.

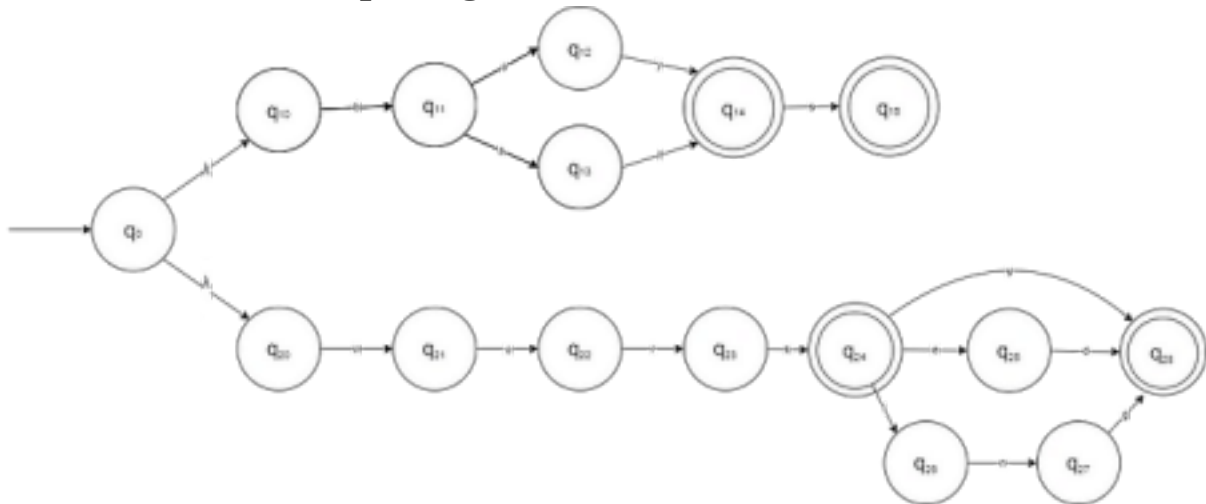


NFA for the words 'boy' and 'bat'

For each category of words, we can form a separate NFA and then combine them using transitions. For example, nouns and their plural can be recognized through one NFA and verbs and their different forms can be recognized through another NFA and finally, the two NFA can be combined. The figure below shows the NFA for

some

words and their morphological variations.

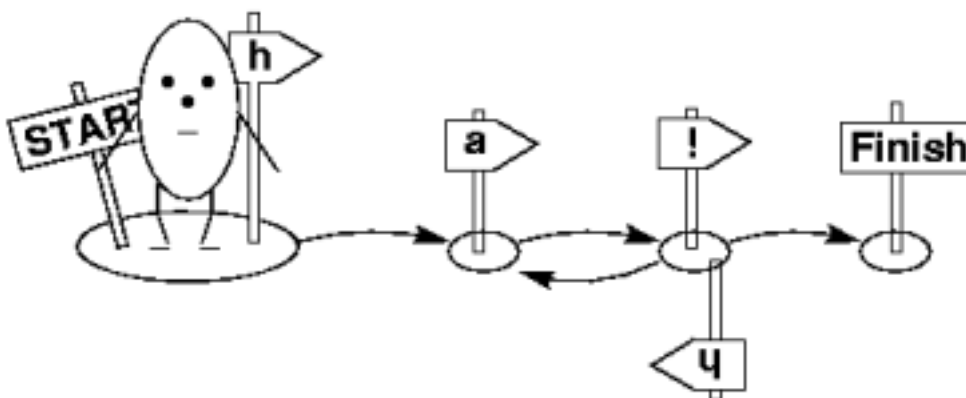


NFA for some words and their morphological variations

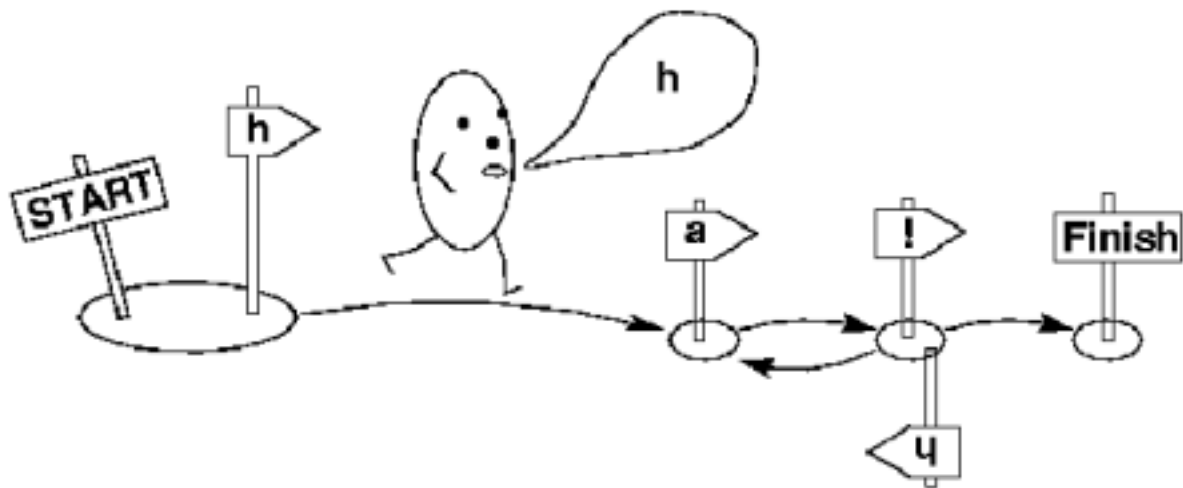
A Simple Machine that can laugh

A finite state generator is a simple computing machine that outputs a sequence of symbols.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material



It starts in some start state and then tries to reach a final state by making transitions from one state to another. Every time it makes such a transition it *emits* (or *writes* or *generates*) a symbol.

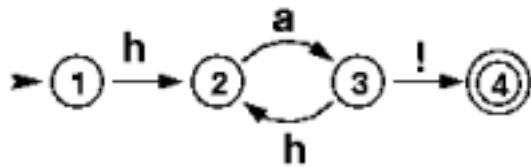


So, what does the generator in the pictures say? It laughs. It generates sequences of symbols of the form ha! or haha! or hahaha! or hahahaha! and so on. Why does it behave like that? Well, it first has to make a transition emitting h. The state that it reaches through this transition is not a final state. So, it has to keep on going emitting an a. Here, it has two possibilities: it can either follow the ! arrow, emitting ! and then stopping in the final state (but remember, it can't look ahead to see that it would reach a final state with the ! transition)

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material
or it can follow the h arrow emitting an h and going back to the state where it just came from.

Finite state generators can be thought of as directed graphs. And in fact finite state generators are usually drawn as directed graphs. Here is our laughing machine as we will from now on draw finite state

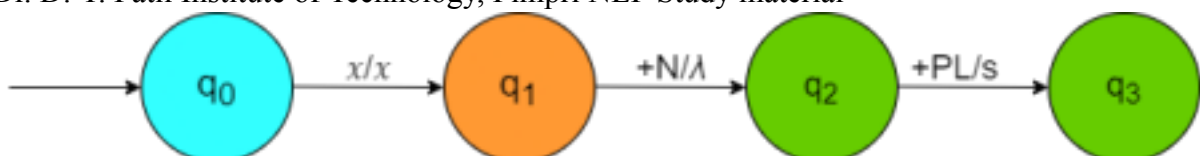
generators:



Morphological parsing and generation

Morphological parsing is the process of producing a lexical structure of a word, that is, breaking a word into stem and affixes and labeling the morphemes with category label. For example the word ‘books’ can be parsed as book+s and further as book + N+PL, the word ‘went’ can be parsed as go+V+PAST etc. generation is the reverse process of parsing, that is combining the lexical form of a word to produce the word. For example, box+N+Pl generates the word ‘boxes’. Finite state transducers are quite useful in morphological parsing. Let us consider the lexicon form of regular inflectional ‘girl +N +PL’. Figure below shows the transducer which convert the lexicon form ‘girl+N+PL’ into the word ‘girls’. Let us assume that x represents the word ‘girl’ for simplification purpose as for every regular singular noun the input and output will remain same in a transducer, that’s why a variable x can be used for a noun. The word ‘girl’ can be replaced by any other regular noun like ‘boy’.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material



Generation of word from its lexical form through a transducer

Here simple regular nouns have been considered on exemplarily basis, orthographic rules can be applied for irregular inflections and

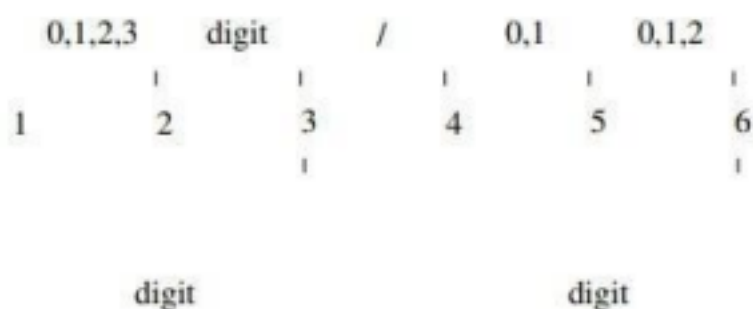
derivations and NFAs can be designed for every word and its variations in a certain language.

In other sense, we can say that morphology is the study of – ► The formation of words. ► The origin of the words. ► Grammatical forms of the words. ► Use of prefixes and suffixes in the formation of words. ► How parts-of-speech (PoS) of a language are formed.

Finite state automata for recognition

The approach to spelling rules that is described here involves the use of finite state transducers (FSTs). Rather than jumping straight into this, we will briefly consider the simpler finite state automata and how they can be used in a simple recogniser. Suppose we want to recognise dates (just day and month pairs) written in the format day/month. The day and the month may be expressed as one or two digits (e.g. 11/2, 1/12 etc). This format corresponds to the following simple FSA, where each character corresponds to one transition:

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material



This is a non-deterministic FSA: for instance, an input starting with the digit 3 will move the FSA to both state 2 and state 3. This

corresponds to a local ambiguity: i.e., one that will be resolved by subsequent context. By convention, there must be no 'left over' characters when the system is in the final state.

- **Finite-State Transducers (FSTs) in Morphological Parsing:**

1. Definition of Finite-State Transducers:

- An FST is a computational model that describes a mapping between input and output sequences. In the context of morphological parsing, FSTs are employed to represent the relationships between surface forms (word forms) and their underlying morphological structures.

2. Components of an FST for Morphological Parsing:

- *States*: Represent different points in the morphological analysis process.
- *Transitions*: Define the relationships between states and the input/output symbols.
- *Input Alphabet*: Consists of symbols representing the surface forms of words.
- *Output Alphabet*: Contains symbols representing the morphological structures or features associated with the words.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

3. Construction of FSTs for Morphological Parsing:

- *Lexicon*: FSTs are constructed based on linguistic resources like lexicons and morphological dictionaries. Each entry in the lexicon corresponds to a state in the FST.
- *Affixes*: FSTs include transitions for affixes (prefixes, suffixes) and morphological features. These transitions connect states based on the morphological transformations.

4. Morphological Analysis with FSTs:

- Given a surface form (word), the FST is used to traverse through its states and transitions, providing a sequence of output symbols that represent the morphological analysis of the word.
- For example, if the surface form is "running," the FST may generate an output sequence like "run + V + ing," indicating the lemma "run," the part-of-speech "verb," and the morphological feature "present participle."

5. Ambiguity Handling:

- FSTs can handle ambiguity in morphological parsing by encoding multiple possible analyses. The FST can represent alternative paths through the states, each corresponding to a different interpretation of the word's morphology.

6. Weighted Transitions:

- In some cases, weights can be assigned to transitions to represent probabilities or preferences for specific morphological analyses. This is especially useful in situations where multiple analyses are possible, and one needs to be chosen based on likelihood.

7. Integration with NLP Applications:

- FSTs for morphological parsing are integrated into larger NLP applications, such as part-of-speech tagging, machine translation,

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material and information retrieval. They provide essential morphological insights for downstream language processing tasks.

8. Optimization and Efficiency:

- Techniques like determinization and minimization are applied to optimize FSTs for efficiency in terms of memory usage and parsing speed.

9. Applications Beyond Morphological Parsing:

- While FSTs are widely used for morphological parsing, they also find applications in other linguistic tasks, including phonology, syntax, and spell checking.

In summary, Finite-State Transducers are powerful tools for modeling the morphological structure of words in natural language. They provide a flexible framework for representing linguistic knowledge, enabling efficient and accurate morphological parsing in various NLP applications.

• Transducers and Orthographic rules,

The problem of recognizing that foxes breaks down into the two morphemes fox and -es is called morphological parsing. • Similar problem in the information retrieval domain: stemming • Given the surface or input form going, we might want to produce the parsed form: VERB-go + GERUND-ing • In this chapter – morphological knowledge and – The finite-state transducer • It is quite inefficient to list all forms of noun and verb in the dictionary because the productivity of the forms. • Morphological parsing is necessary for more than just IR(Information Retrieval), but also – Machine translation – Spelling checking
Survey of (Mostly) English Morphology

- Morphology is the study of the way words are built up from smaller meaning-bearing units, morphemes.
- Two broad classes of morphemes: – The stems: the “main” morpheme of the word, supplying the main meaning, while – The affixes: add “additional” meaning of various kinds.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- Affixes are further divided into prefixes, suffixes, infixes, and circumfixes. – Suffix: eat-s – Prefix: un-buckle – Circumfix: ge-sag-t (said) sagen (to say) (in German) – Infix: hingi (borrow) humingi (the agent of an action))in Philippine language Tagalog) Prefixes and suffixes are often called concatenative morphology. • A number of languages have extensive non-concatenative morphology – The Tagalog infixation example – Templatic morphology or root-and-pattern morphology, common in Arabic, Hebrew, and other Semitic languages • Two broad classes of ways to form words from morphemes: – Inflection: the combination of a

word stem with a grammatical morpheme, usually resulting in a word of the same class as the original stem, and usually filling some syntactic function like agreement, and – Derivation: the combination of a word stem with a grammatical morpheme, usually resulting in a word of a different class, often with a meaning hard to predict exactly In English, only nouns, verbs, and sometimes adjectives can be inflected, and the number of affixes is quite small. •

Inflections of nouns in English: – An affix marking plural, • cat(-s), thrush(-es), ox (oxen), mouse (mice) • ibis(-es), waltz(-es), finch(-es), box(-es), butterfly(-ies) – An affix marking possessive • llama's, children's, llamas', Euripides' comedies Verbal inflection is more complicated than nominal inflection. – English has three kinds of verbs: • Main verbs, eat, sleep, impeach • Modal verbs, can will, should • Primary verbs, be, have, do – Morphological forms of regular verbs

These regular verbs and forms are significant in the morphology of English because of their majority and being productive.

Morphological forms of irregular verbs

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Nominalization in English: – The formation of new nouns, often from verbs or adjectives

Derivation in English is more complex than inflection because – Generally less productive • A nominalizing affix like –ation can not be added to absolutely every verb. eatation(*) – There are subtle and complex meaning differences among nominalizing suffixes. For example, sincerity has a subtle difference in meaning from sincereness.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material
Finite-State Morphological Parsing

We need at least the following to build a morphological parser:

1. Lexicon: the list of stems and affixes, together with basic information about them (Noun stem or Verb stem, etc.)
2. Morphotactics: the model of morpheme ordering that explains which classes of morphemes can follow other classes of morphemes inside a word. E.g., the rule that English plural morpheme follows the noun rather than preceding it.
3. Orthographic rules: these spelling rules are used to model the changes that occur in a word,

usually when two morphemes combine (e.g., the y→ie spelling rule changes city + -s to citie

Finite-State Morphological Parsing The Lexicon and Morphotactics

A lexicon is a repository for words. – The simplest one would consist of an explicit list of every word of the language. Inconvenient or impossible! – Computational lexicons are usually structured with • a list of each of the stems and • Affixes of the language together with a representation of morphotactics telling us how they can fit together. – The most common way of modeling morphotactics is the finite-state automaton.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

English derivational morphology is more complex than English inflectional morphology, and so automata of modeling English derivation tends to be quite complex. – Some even based on CFG • A small part of morphosyntactics of English adjectives

Finite-State Morphological Parsing

An FSA for a fragment of English adjective Morphology #2 • The FSA#1 recognizes all the listed adjectives, and ungrammatical forms like unbig, redly, and realest. • Thus #1 is revised to become #2. • The complexity is expected from English derivation.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

We can now use these FSAs to solve the problem of morphological recognition: –
Determining whether an input string of letters makes up a legitimate English word or not –
We do this by taking the morphotactic FSAs, and plugging in each “sub-lexicon” into the FSA. – The resulting FSA can then be defined as the level of the individual letter.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Finite-State Morphological Parsing Morphological Parsing with FST

- Given the input, for example, cats, we would like to produce cat +N +PL.
- Two-level morphology, by Koskenniemi (1983) – Representing a word as a correspondence between a lexical level
- Representing a simple concatenation of morphemes making up a word, and – The surface level
- Representing the actual spelling of the final word.
- Morphological parsing is implemented by building mapping rules that maps letter sequences like cats on the surface level into morpheme and features sequence like cat +N +PL on the lexical level.

The automaton we use for performing the mapping between these two levels is the finite-state transducer or FST

- . – A transducer maps between one set of symbols and another;
- An FST does this via a finite automaton.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- Thus an FST can be seen as a two-tape automaton which recognizes or generates pairs of strings.

- The FST has a more general function than an FSA: – An FSA defines a formal language – An FST defines a relation between sets of strings

- Another view of an FST: – A machine reads one string and generates another.

FST as recognizer:

- a transducer that takes a pair of strings as input and output accept if the string-pair is in the string-pair language, and a reject if it is not.

- FST as generator:

- a machine that outputs pairs of strings of the language. Thus the output is a yes or no, and a pair of output strings.

- FST as transducer:

- A machine that reads a string and outputs another string.

- FST as set relater:

- A machine that computes relation between sets.

A formal definition of FST (based on the Mealy machine extension to a simple FSA)

- Q : a finite set of N states $q_0, q_1, \dots, q_N$

- Σ : a finite alphabet of complex symbols. Each complex symbol is composed of an input output pair $i : o$; one symbol I from an input alphabet I , and one symbol o from an output alphabet O , thus $\Sigma \subseteq I \times O$. I and O may each also include the epsilon symbol ϵ . – q_0 : the start state

- F : the set of final states, $F \subseteq Q$

- $\delta(q, i:o)$: the transition function or transition matrix between states. Given a state $q \in Q$ and complex symbol $i:o \in \Sigma$, $\delta(q, i:o)$ returns a new state $q' \in Q$. δ is thus a relation from $Q \times \Sigma$ to Q .

FSAs are isomorphic to regular languages,

FSTs are isomorphic to regular relations

- Regular relations are sets of pairs of strings, a natural extension of the regular language, which are sets of strings.

- FSTs are closed under union, but generally they are not closed under difference, complementation, and intersection.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- Two useful closure properties of FSTs:
 - Inversion: If T maps from I to O , then the inverse of T , T^{-1} maps from O to I . –
 - Composition: If T_1 is a transducer from I_1 to O_1 and T_2 a transducer from I_2 to O_2 , then $T_1 \circ T_2$ maps from I_1 to O_2

Inversion is useful because it makes it easy to convert a FST-as-parser into an FST-as-generator. • Composition is useful because it allows us to take two transducers than run in series and replace them with one complex transducer. – $T_1 \circ T_2(S) = T_2(T_1(S))$

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- **The Porter Stemmer,**

Information retrieval

- One of the mostly widely used stemming algorithms is the simple and efficient Porter (1980) algorithm, which is based on a series of simple cascaded rewrite rules.

- ATIONAL → ATE (e.g., relational → relate) – ING → eif stem contains vowel (e.g., motoring → motor)

- Problem: – Not perfect: error of commision, omission
- Experiments have been made – Some improvement with smaller documents – Any improvement is quite small

• Porter Stemmer

Types of Stemmer in NLTK

Python NLTK contains a variety of stemming algorithms, including several types. Let's examine them down below.

1. Porter's Stemmer

It is one of the most popular stemming methods proposed in 1980. It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes. This stemmer is known for its speed and simplicity. The main applications of Porter Stemmer include data mining and Information retrieval. However, its applications are only limited to English words. Also, the group of stems is mapped on to the same stem and the output stem is not necessarily a

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material
meaningful word. The algorithms are fairly lengthy in nature and are known to be the oldest stemmer.

Example: EED -> EE means "if the word has at least one vowel and consonant plus EED ending, change the ending to EE" as 'agreed' becomes 'agree'.

Implementation of Porter Stemmer

```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer instance
porter_stemmer = PorterStemmer()

# Example words for stemming
words = ["running", "jumps", "happily", "running", "happily"]

# Apply stemming to each word
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print the results
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Output:

Original words: ['running', 'jumps', 'happily', 'running', 'happily']

Stemmed words: ['run', 'jump', 'happili', 'run', 'happili']

Advantage: It produces the best output as compared to other stemmers and it has less error rate.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Limitation: Morphological variants produced are not always real words.

2. Lovins Stemmer

It is proposed by Lovins in 1968, that removes the longest suffix from a word then the word is recorded to convert this

stem into valid words.

Example: sitting -> sitt -> sit

Advantage: It is fast and handles irregular plurals like ‘teeth’ and ‘tooth’ etc.

Limitation: It is time consuming and frequently fails to form words from stem.

3. Dawson Stemmer

It is an extension of Lovins stemmer in which suffixes are stored in the reversed order indexed by their length and last letter.

Advantage: It is fast in execution and covers more suffices.

Limitation: It is very complex to implement.

4. Krovetz Stemmer

It was proposed in 1993 by Robert Krovetz. Following are the steps:

- 1) Convert the plural form of a word to its singular form.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- 2) Convert the past tense of a word to its present tense and remove the suffix ‘ing’.

Example: ‘children’ -> ‘child’

Advantage: It is light in nature and can be used as pre-stemmer for other stemmers.

Limitation: It is inefficient in case of large documents.

5. Xerox Stemmer

Capable of processing extensive datasets and generating valid words, it has a tendency to over-stem, primarily due to its reliance on lexicons, making it language-dependent. This constraint implies that its effectiveness is limited to specific languages.

Example:

‘children’ -> ‘child’

‘understood’ -> ‘understand’

‘whom’ -> ‘who’

‘best’ -> ‘good’

6. N-Gram Stemmer

The algorithm, aptly named n-grams (typically $n=2$ or 3), involves breaking words into segments of length n and then applying statistical analysis to identify patterns. An n-gram is a set of n consecutive characters extracted from a word in which similar words will have a high proportion of n-grams in common.

Example: 'INTRODUCTIONS' for $n=2$ becomes : *I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S*

Advantage: It is based on string comparisons and it is language dependent.

Limitation: It requires space to create and index the n-grams and it is not time efficient.

7. Snowball Stemmer

The Snowball Stemmer, compared to the Porter Stemmer, is multi-lingual as it can handle non-English words. It supports various languages and is based on the 'Snowball' programming language, known for efficient processing of small strings.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

The Snowball stemmer is way more aggressive than Porter Stemmer and is also referred to as Porter2 Stemmer. Because of the improvements added when compared to the Porter Stemmer, the Snowball stemmer is having greater computational speed.

Implementation of Snowball Stemmer

```
from nltk.stem import SnowballStemmer

# Choose a language for stemming, for example, English
stemmer = SnowballStemmer(language='english')

# Example words to stem
words_to_stem = ['running', 'jumped', 'happily', 'quickly', 'foxes']

# Apply Snowball Stemmer
stemmed_words = [stemmer.stem(word) for word in words_to_stem]

# Print the results
print("Original words:", words_to_stem)
print("Stemmed words:", stemmed_words)
```

Output:

Original words: ['running', 'jumped', 'happily', 'quickly', 'foxes']
Stemmed words: ['run', 'jump', 'happili', 'quick', 'fox']

8. Lancaster Stemmer

The Lancaster stemmers are more aggressive and dynamic

compared to the other two stemmers. The stemmer is really faster, but the algorithm is really confusing when dealing with small words. But they are not as efficient as Snowball

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Stemmers. The Lancaster stemmers save the rules externally and basically uses an iterative algorithm.

Implementation of Lancaster Stemmer

```
from nltk.stem import LancasterStemmer

# Create a Lancaster Stemmer instance
stemmer = LancasterStemmer()

# Example words to stem
words_to_stem = ['running', 'jumped', 'happily', 'quickly', 'foxes']

# Apply Lancaster Stemmer
stemmed_words = [stemmer.stem(word) for word in words_to_stem]

# Print the results
print("Original words:", words_to_stem)
print("Stemmed words:", stemmed_words)
```

• Word and Sentence Tokenization,

What is tokenization?

Tokenization is the process of breaking down a piece of text, like a sentence or a paragraph, into individual words or “tokens.” These tokens are the basic building blocks of language, and tokenization helps computers understand and process human language by splitting it into manageable units.

For example, tokenizing the sentence “I love ice cream” would result in three tokens: “I,” “love,” and “ice cream.” It’s a fundamental step in natural language

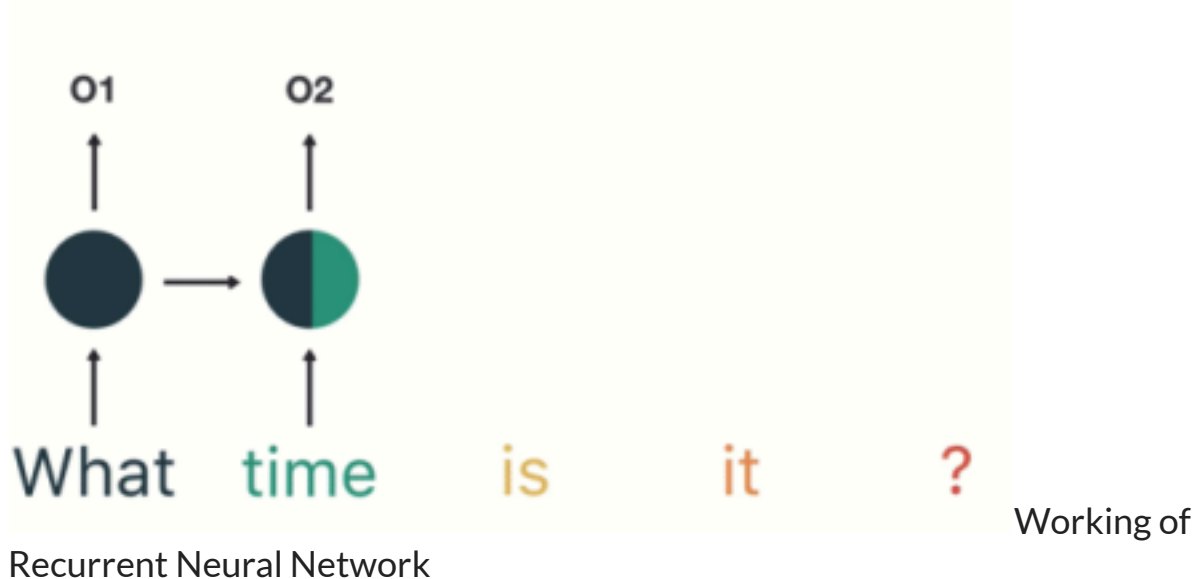
processing and text analysis tasks.

The True Reasons behind Tokenization

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

As tokens are the building blocks of Natural Language, the most common way of processing the raw text happens at the token level.

For example, Transformer based models – the State of The Art (SOTA) Deep Learning architectures in NLP – process the raw text at the token level. Similarly, the most popular deep learning architectures for NLP like RNN, GRU, and LSTM also process the raw text at the token level.



As shown here, RNN receives and processes each token at a particular timestep.

Hence, Tokenization is the foremost step while modeling text data. Tokenization is performed on the corpus to obtain tokens. The following tokens are then used to prepare a vocabulary. Vocabulary refers to the set of unique tokens in the corpus. Remember that vocabulary can be constructed

by considering each unique token in the corpus or by considering the top K Frequently Occurring Words.

Creating Vocabulary is the ultimate goal of Tokenization.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

One of the simplest hacks to boost the performance of the NLP model is to create a vocabulary out of top K frequently occurring words.

Now, let's understand the usage of the vocabulary in Traditional and Advanced Deep Learning-based NLP methods.

- Traditional NLP approaches such as Count Vectorizer and TF-IDF use vocabulary as features. Each word in the vocabulary is treated as a unique feature:

	I	play	cricket	football	tennis
Doc 1	1	1	1	1	1
Doc 2	1	1	0	1	0
Doc 3	0	1	1	0	0
Doc 4	1	1	0	0	1

Traditional NLP: Count Vectorizer

- In Advanced Deep Learning-based NLP architectures, vocabulary is used to create the tokenized input sentences. Finally, the tokens of these sentences are passed as inputs to the model

Which Tokenization Should you use?

As discussed earlier, tokenization can be performed on word, character, or

subword level. It's a common question – which Tokenization should we use while solving an NLP task? Let's address this question here.

Word Tokenization

Word Tokenization is the most commonly used tokenization algorithm. It splits a piece of text into individual words based on a certain delimiter.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Depending upon delimiters, different word-level tokens are formed.

[Pretrained Word Embeddings](#) such as Word2Vec and GloVe comes under word tokenization.

But, there are few drawbacks to this.

Drawbacks of Word Tokenization

One of the major issues with word tokens is dealing with **Out Of Vocabulary (OOV) words**. OOV words refer to the new words which are encountered at testing. These new words do not exist in the vocabulary. Hence, these methods fail in handling OOV words.

But wait – don't jump to any conclusions yet!

- A small trick can rescue word tokenizers from OOV words. The trick is to form the vocabulary with the Top K Frequent Words and replace the rare words in training data with **unknown tokens (UNK)**. This helps the model to learn the representation of OOV words in terms of UNK tokens
- So, during test time, any word that is not present in the vocabulary will be mapped to a UNK token. This is how we can tackle the problem of OOV in word tokenizers.
- The problem with this approach is that the entire information of the

word is lost as we are mapping OOV to UNK tokens. The structure of the word might be helpful in representing the word accurately. And another issue is that every OOV word gets the same representation

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material



Another issue with word tokens is connected to the size of the vocabulary. Generally, pre-trained models are trained on a large volume of the text corpus. So, just imagine building the vocabulary with all the unique words in such a large corpus. This explodes the vocabulary!

This opens the door to Character Tokenization.

Character Tokenization

Character Tokenization splits a piece of text into a set of characters. It overcomes the drawbacks we saw above about Word Tokenization.

- Character Tokenizers handles OOV words coherently by preserving the information of the word. It breaks down the OOV word into characters

and represents the word in terms of these characters

- It also limits the size of the vocabulary. Want to talk a guess on the size of the vocabulary? 26 since the vocabulary contains a unique set of characters

Drawbacks of Character Tokenization

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

Character tokens solve the OOV problem but the length of the input and output sentences increases rapidly as we are representing a sentence as a sequence of characters. As a result, it becomes challenging to learn the relationship between the characters to form meaningful words.

• Detecting and Correcting Spelling Errors,

The noisy channel approach can also be applied to detect and correct real-word real-word error detection spelling errors, errors that result in an actual word of English. This can happen from typographical errors (insertion, deletion, transposition) that accidentally produce a real word (e.g., there for three) or because the writer substituted the wrong spelling of a homophone or near-homophone (e.g., dessert for desert, or piece for peace). A number of studies suggest that between 25% and 40% of spelling errors are valid English words as in the following examples (Kukich, 1992):

This used to belong to thew queen.

They are leaving in about fifteen minuets to go to her house.

The design an construction of the system will take more than a year.

Can they lave him my messages?

The study was conducted mainly be John Black.

The noisy channel can deal with real-word errors as well. Let's begin with a version of the noisy channel model first proposed by Mays et al. (1991) to deal with these real-word spelling errors. Their algorithm takes the input sentence $X = \{x_1, x_2, \dots, x_k, \dots, x_n\}$, generates a large set of candidate correction sentences $C(X)$, then picks the sentence with the highest language model probability.

To generate the candidate correction sentences, we start by generating a set of candidate

words for each input word x_i . The candidates, $C(x_i)$, include every English word with a small edit distance from x_i . With edit distance 1, a common choice (Mays et al., 1991), the candidate set for the real word error *thew* (a rare word meaning ‘muscular strength’) might be $C(\text{thew}) = \{\text{the, thaw, threw, them, thwe}\}$. We then make the simplifying assumption that every sentence has only one error. Thus the set of candidate sentences $C(X)$ for a sentence X = Only two of thew apples would be:

8 APPENDIX B • SPELLING CORRECTION AND THE NOISY CHANNEL

only two of thew apples oily two of thew apples only too of thew apples only to

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

of thew apples only tao of the apples only two on thew apples only two off thew apples only two of the apples only two of threw apples only two of thew applies only two of thew dapples

• Minimum Edit Distance,

Many NLP tasks are concerned with measuring how similar two strings are. • Spell correction:

- The user typed “graffe”
- Which is closest? : graf grail giraffe
- the word giraffe, which differs by only one letter from graffe, seems intuitively to be more similar than, say grail or graf,
- The minimum edit distance between two strings is defined as the minimum number of editing operations (insertion, deletion, substitution) needed to transform one string into another The minimum edit distance between intention and execution can be visualized using

their **alignment**.

- Given two sequences, an alignment is a correspondence between substrings of the two sequences.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

If each operation has cost of 1

- Distance between them is 5
- If substitutions cost 2 (Levenshtein Distance)
 - Distance between them is 8

Minimum **Edit Distance Algorithm**

How do we find the minimum edit distance?

- We can think of this as a search task, in which we are searching for the shortest path
 - a sequence of edits
 - from one string to another.

- The space of all possible edits is enormous, so we can't search naively . – Most of distinct

edit paths ends up in the same state, so rather than recomputing all those paths, we could just remember the shortest path to a state each time we saw it. – We can do this by using dynamic programming.

– Dynamic programming is the name for a class of algorithms that apply a table-driven method to solve problems by combining solutions to sub-problems.

Minimum Edit Distance between Two Strings

For two strings

- the source string X of length n
- the target string Y of length m

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

- We define $D(i,j)$ as the edit distance between $X[1..i]$ and $Y[1..j]$
- i.e., the first i characters of X and the first j characters of Y
- The edit distance between X and Y is thus $D(n,m)$

➤ Human Morphological Processing

N-Grams: • Building N-gram for spelling corrections,

N-grams are contiguous sequences of N items (words, characters, or subword units) extracted from a given text. In the context of spelling corrections, N-grams help identify and correct misspelled words by comparing them to correctly spelled N-gram patterns.

Process:

1. Corpus Collection:

- Collect a large corpus of correctly spelled text data to build the foundation for creating N-grams. This corpus serves as a reference for identifying valid word patterns.

2. N-gram Generation:

- Break down the text into N-grams. For spelling corrections, typically, character level N-grams are used. For example, in the word "apple," 2-grams (bigrams) would be ['ap', 'pp', 'pl', 'le'].

3. Frequency Count:

- Calculate the frequency of each N-gram in the corpus. This step helps in determining the likelihood or commonality of each N-gram.

4. Building the Model:

- Create a spelling correction model by associating correct N-grams with their frequencies. This model can be used to suggest corrections for misspelled words by identifying similar N-grams in the input.

5. Correction Suggestions:

- When a misspelled word is encountered, break it into N-grams, compare them to the model, and suggest corrections based on the most probable N-grams found in the corpus.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

2. N-gram for Language Model:

Definition:

- N-gram language models estimate the likelihood of a word or sequence of words given the context of the preceding N-1 words. These models are used to predict the next word in a sentence and aid in various natural language processing tasks.

Process:

1. Corpus Collection:

- Gather a diverse and representative corpus of text data to train the language model. The larger and more varied the corpus, the better the language model's generalization.

2. Tokenization:

- Tokenize the text into words or subword units. This step is essential for accurately building N-grams at the word level.

3. N-gram Generation:

- Create N-grams by considering contiguous sequences of N words. For example, in the sentence "The cat sat on the mat," 2-grams (bigrams) would be ['The cat', 'cat sat', 'sat on', 'on the', 'the mat'].

4. Frequency Count:

- Count the occurrences of each N-gram in the corpus. This step helps estimate the probability of encountering a particular sequence of words.

5. Language Model Construction:

- Build the language model by associating N-grams with their probabilities. This model allows for the prediction of the likelihood of a word given its context.

6. Predictive Use:

- Utilize the trained language model for various NLP tasks, such as speech recognition, machine translation, and text generation. The model predicts the next word based on the context provided by the preceding N-1 words.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material

ASSIGNMENT QUESTIONS

1. English and Indian Languages:

- Compare and contrast the morphological characteristics of English and Indian languages. Highlight key differences in terms of word structures and morphological processes.

Finite-State Morphological Parsing:

2. Building a Finite-State Lexicon:

- Outline the steps involved in building a finite-state lexicon for a natural language. Discuss the importance of a lexicon in finite-state morphological parsing.

3. Finite-State Transducers (FSTs) for Morphological Parsing:

- Explain how Finite-State Transducers are utilized in morphological parsing. Provide examples to illustrate how FSTs can model morphological processes in languages.

4. Transducers and Orthographic Rules:

- Investigate the role of transducers in implementing orthographic rules in natural language processing. Provide examples of how transducers can handle orthographic variations.

5. The Porter Stemmer:

- Explore the Porter Stemmer algorithm and its significance in stemming. Discuss the principles behind the Porter Stemmer and its applications in information retrieval.

6. Word and Sentence Tokenization:

- Define and compare word and sentence tokenization. Discuss the challenges associated with tokenization in different languages and how it impacts downstream NLP tasks.

7. Detecting and Correcting Spelling Errors:

- Design a system for detecting and correcting spelling errors. Discuss the role of language models, N-grams, and other techniques in building an effective spelling correction system.

8. Minimum Edit Distance:

- Explain the concept of minimum edit distance in the context of spelling correction. Discuss algorithms such as Levenshtein distance and their applications in measuring the similarity between strings.

Dr. D. Y. Patil Institute of Technology, Pimpri NLP Study material
Human Morphological Processing N-Grams:

9. Building N-gram for Spelling Corrections:

- Elaborate on the process of building N-grams for spelling corrections. Discuss how N-grams can be used to identify and correct misspelled words in natural language text.

10. N-gram for Language Model:

- Investigate the role of N-grams in language modeling. Discuss how N-gram language models are constructed, trained, and used for predicting the likelihood of word sequences in a given context.

